Agent Memory System Card (AMS Card)

Contents

1 Directory **3**

2 Main Card **4**

AMS Card Template for *[Memory System Name]* 4
How To Use This Template 4
System Overview 4
System and Deployment Details 5
Intended Use 5
Core Memory Capabilities 5
Memory Form and Architecture 6
Memory Lifecycle 7
Performance and Evaluation 8
Limitations and Risks 9
Maintenance and Governance 9
Privacy, Sovereignty, and Safety 9
Citation 11
Glossary 11
More Information 12

3 Structured Index Record **12**

1 Directory

The AMS Card is designed to serve multiple stakeholders. Depending on your role, you may want to focus on different sections to find the information most relevant to you.

End Users

End users should focus on sections that explain what the system remembers, how it impacts their experience, and what control they have over their data.

- **System Overview:** Summarizes the available evidence about performance, contextual integrity, user control, portability, and data governance.
- **Intended Use:** Explains how the memory system is meant to be used directly and outlines any out-of-scope or unsupported uses.
- **Core Memory Capabilities:** Details the exact types of information the system captures, such as user facts, preferences, or task history.
- **Limitations and Risks:** Highlights potential behavioral risks you might encounter, such as false memories, stale information, or inappropriate personalization.
- **Privacy, Sovereignty, and Safety:** Covers critical information about transparency, user control, editability, and how your sensitive data is handled.

Auditors (Compliance & Governance)

Auditors, privacy reviewers, and governance teams should focus on the sections detailing evidence, compliance metrics, and operational safeguards.

- **System Overview:** Gives short, evidence-linked answers about performance, contextual integrity, user controls, portability, and data residency.
- **Performance and Evaluation:** Details the empirical evidence supporting the system's claims, including the evaluation setup, datasets used, and specific performance metrics.
- **Maintenance and Governance:** Outlines audit logging practices, runtime monitoring, user feedback channels, and retention policies.
- **Privacy, Sovereignty, and Safety:** Provides an extended treatment of contextual integrity, PII detection, security threat models, and adversarial robustness.
- **Structured Index Record:** Provides typed and controlled fields for machine indexing and cross-system comparison.

Developers & System Integrators

Developers and engineers integrating or modifying the system should focus on the technical architecture, implementation details, and lifecycle mechanics.

- **Structured Index Record:** Lists supported integration interfaces, modalities, and memory forms using controlled values.
- **Memory Form and Architecture:** Explains the underlying data representation (such as vector embeddings or graphs), storage locations, and the technical read/write paths.

- **Memory Lifecycle:** Describes the mechanical processes of how information is captured, consolidated, forgotten, and retrieved by the agent or model.
- **Model and Dependency Information:** Lists the required base models, embedding models, databases, and external services necessary to run the system.

2 Main Card

The main card provides the human-readable description and evidence that cannot be inferred from structured fields alone. It covers the system and its intended use, its core memory capabilities and their boundaries, its representational form and architecture, the memory lifecycle of capture, evolution, and retrieval, the performance and efficiency evidence, the limitations and risks, the maintenance and governance posture, and an extended treatment of privacy, sovereignty, and safety, before closing with a citation block and a glossary. Throughout, *[bracketed italic text]* marks a field to be filled in by the author of a specific card, `[More Information Needed]` marks an unknown or not-yet-evaluated field, and `Not Applicable` marks a field that does not apply to a given memory system design.

AMS Card Template for *[Memory System Name]*

This template documents a memory system for AI agents, memory-augmented models, or applications that preserve and reuse information over time. It is organized around the system's metadata, intended use, memory capabilities, architecture, lifecycle, performance, limitations, maintenance, and privacy properties.

Use `[More Information Needed]` when a field is unknown or not yet evaluated. Use `Not Applicable` when a field does not apply to the system design. The template should work for a broad range of memory systems, including raw-text stores, RAG-style retrieval systems, vector databases, graph memories, profile stores, latent memories, parametric memories, multimodal memories, and hybrid architectures.

How To Use This Template

Use this template as shared documentation for everyone involved in building, integrating, evaluating, deploying, governing, or interacting with a memory system. It should make clear what the memory system remembers, how memory is represented, how information enters and leaves the system, how memory affects behavior, which performance claims are supported, and which operational or privacy risks remain.

Start with the System Overview, using short prose and links or citations to summarize the evidence most relevant to users and reviewers. Then describe the implementation, interfaces, memory capabilities, architecture, lifecycle, and dependencies before documenting performance, limitations, maintenance, governance, and privacy in detail. Complete the Structured Index Record at the end for discovery and comparison. If a section does not apply to the system, state why rather than forcing the implementation into a category it does not use.

Do not assume that every memory system has explicit working memory, experiential memory, retrieval, human-readable storage, external storage, or editability. Describe the actual behavior first, then use the definitions in this template to classify it where useful.

System Overview

Answer the following questions in short prose. Link to tests, evaluations, audits, product documentation, or other supporting material where available, and distinguish intended behavior from observed evidence. Use `[No public evidence identified]` when a claim cannot be supported from available information.

System summary: *[What is the system, and what is it designed to remember?]*

Performance evidence: *[What evidence shows that the system improves performance for its intended use?]*
Contextual integrity evidence: *[What evidence shows that the system prevents information from crossing inappropriate user, application, workspace, tenant, or other operational contexts?]*
Inspection, editing, and deletion: *[How can users inspect, edit, and permanently delete memories?]*
Memory preferences: *[How can users control what information the system does and does not remember?]*
Portability: *[How can users export their memories when switching memory systems?]*
Data governance and residency: *[What control do users or administrators have over where memories are physically stored?]*

System and Deployment Details

Deployment Context

Shared by: *[Organization or individual, if different]*
System type: *[Memory framework | memory backend | model adapter | model fine-tune | agent scaffold | database | browser memory | on-device memory | other]*
Deployment mode: *[Local | on-device | self-hosted | cloud-hosted | managed service | embedded in model weights | hybrid]*
Primary users: *[Developers | researchers | end users | enterprise administrators | agent builders | other]*
Primary affected users: *[Whose data may be stored, transformed, retrieved, or exposed through this system?]*

Sources

Repository: *[URL]*
Documentation: *[URL]*
Technical reference: *[URL or citation]*
Demo or service endpoint: *[URL]*
Changelog: *[URL]*

Intended Use

Direct Use

[Describe how the memory system is intended to be used directly, without substantial modification. Include intended agent types, interaction settings, data sources, and user workflows.]

Downstream Use

[Describe how the memory system may be integrated into larger products, agents, model-serving stacks, personalization systems, research benchmarks, or enterprise workflows.]

Out-of-Scope Use

[Describe uses that are unsupported, unsafe, untested, or outside the system's design intent. Include domains, data types, deployment settings, or user groups for which the system should not be used without additional evaluation.]

Core Memory Capabilities

Describe what the system is meant to remember or make available for future use. Use the examples below only where they fit; do not force the system into a category it does not implement.

Remembered Information

Information types: *[User facts | preferences | environment state | task history | tool outputs | documents | embeddings | m activations | updated model parameters | skills | strategies | policies | other]*

Temporal scope: *[Single turn | session | cross-session | long-term | lifelong | periodically reset | other]*
Granularity: *[Raw messages | chunks | facts | entities | relations | summaries | profiles | trajectories | skills | latent states | parameter updates | other]*
Persistence: *[Ephemeral | persistent until deletion | time-limited | policy-controlled | embedded in model weights | other]*

Functional Role

[Explain what the memory enables. Examples include personalization, factual recall, task continuity, long-horizon reasoning, environment state tracking, preference learning, strategy improvement, skill learning, context compression, reduced latency, or model adaptation.]

Optional descriptors:

- **Factual memory:** Stores declarative knowledge such as user facts, preferences, environment facts, documents, or world state.
- **Experiential memory:** Stores or derives lessons from past trajectories, successes, failures, tool-use patterns, cases, skills, or strategies.
- **Working memory:** Maintains task-local context for active reasoning, planning, or state tracking during an ongoing task.
- **Parametric memory:** Encodes information or capabilities in model parameters, adapters, fine-tunes, or other learned weights.
- **Latent memory:** Represents memory in hidden states, embeddings, KV caches, learned vectors, or other machine-native formats.
- **Retrieval-augmented memory:** Stores external items and retrieves relevant content for a model or agent at inference time.

Capability Boundaries

What the system does remember well: `[More Information Needed]`
What the system does not remember or does not aim to remember: `[More Information Needed]`
Known failure modes: *[Hallucinated memories | stale memories | over-retention | under-retention | poor recall | conflicting facts | privacy leakage | other]*

Memory Form and Architecture

Representation

Describe the primary form in which memory is represented. A system may use more than one.

Representation form: *[Raw text | structured text | key-value entries | relational data | graph | vector embeddings | multimodal embeddings | latent activations | KV cache | model parameters | LoRA/adapters | files | browser state | other]*
Human readability: *[Human-readable | partially human-readable | machine-readable only | not inspectable]*
Addressability: *[By ID | semantic search | graph traversal | key lookup | temporal lookup | profile fields | learned routing | implicit model behavior | other]*
Structure: *[Flat list | chunks | tables | profiles | graph | hierarchy | tree | episodic log | semantic layer | community summaries | learned latent space | model weights | hybrid]*
Storage location: *[Local file | database | vector database | graph database | cloud service | browser storage | encrypted store | model weights | model cache | other]*

Agent or Model Coupling

Integration point: *[Prompt/context injection | tool call | middleware | retrieval API | memory manager | model adapter | fine-tuned weights | KV-cache reuse | planner | browser extension | other]*

Invocation pattern: *[Always on | user-triggered | agent-triggered | model-triggered | policy-triggered | task-boundary update | session-boundary update | asynchronous update | other]*

Read path: *[How memory is accessed or used during inference or action selection.]*

Write/update path: *[How new information enters the memory system or changes model behavior.]*

Separation from agent: *[External component | embedded in harness | embedded in model | managed by service | hybrid]*

Data Sources

Input sources: *[User messages | documents | browser content | tool outputs | API responses | calendar/email/files | enviro observations | agent reasoning traces | task outcomes | human feedback | other]*

Collection triggers: *[Every interaction | selected interactions | explicit save command | tool call | background ingestion | training pipeline | fine-tuning job | other]*

Preprocessing: *[Chunking | summarization | extraction | embedding | redaction | filtering | deduplication | schema mapping | training data preparation | other]*

Modality Support

Supported modalities: *[Text | code | image | audio | video | sensor data | UI state | tool traces | structured records | multimodal embeddings | other]*

Cross-modal alignment: *[Describe whether and how memories from different modalities are linked, retrieved, or transformed.]*

Memory Lifecycle

Describe how information moves through the memory system. Use the formation, evolution, and retrieval taxonomy where useful, but keep the description faithful to the actual system.

Capture or Formation

What gets captured: `[More Information Needed]`

How capture decisions are made: *[Rules | prompts | classifiers | user action | agent decision | learned policy | all inputs | training process | other]*

Transformation before storage or internalization: *[None | summarization | knowledge distillation | structured extraction | embedding | latent encoding | parameter update | compression | redaction | other]*

Quality controls: *[Validation | confidence scoring | deduplication | schema checks | human review | tests | other]*

Maintenance or Evolution

Update behavior: *[Append-only | overwrite | merge | consolidate | fine-tune | adapter update | cache update | profile update | no updates after creation | other]*

Conflict handling: *[Recency | confidence | provenance | user authority | source authority | learned resolver | manual review | unsupported]*

Consolidation: *[Describe whether and how redundant memories are merged, summarized, abstracted, or internalized.]*

Forgetting or pruning: *[Manual deletion | time-based expiration | low-access pruning | low-utility pruning | capacity-based eviction | model unlearning | unsupported | other]*

Versioning: *[Describe whether memory state, model weights, profiles, or entries are versioned.]*

Access, Retrieval, or Use

Access method: *[Direct lookup | semantic retrieval | graph query | profile lookup | temporal query | learned routing | prompt construction | implicit parameter recall | latent reuse | other]*

Retrieval timing: *[At task start | every turn | before tool use | after tool use | on demand | when confidence is low | continuous | not retrieval-based]*
Query construction: *[User query | agent-generated query | fixed template | task metadata | environmental context | hidden state | learned query | not applicable]*
Ranking or selection: *[Similarity | recency | importance | source authority | access policy | graph distance | learned ranker | no ranking | other]*
Post-processing: *[Filtering | re-ranking | aggregation | compression | summarization | citation generation | grounding | policy filtering | no post-processing]*
Output format to agent or model: *[Raw snippets | structured facts | summaries | profile fields | graph paths | embeddings | latent vectors | parameterized behavior | other]*

Performance and Evaluation

Performance Claims

State only claims that are supported by evidence, and distinguish measured results from intended benefits.
Claimed benefits: *[Improved recall | personalization | task success | long-horizon coherence | reduced user repetition | improved planning | faster inference | lower token cost | lower storage cost | other]*
Evidence level: *[Anecdotal | internal tests | benchmark results | user study | production telemetry | external evaluation | peer-reviewed evidence]*
Compared against: *[No memory baseline | context-only baseline | RAG baseline | vector store baseline | previous version | competing system | other]*

Evaluation Setup

Benchmarks or tasks: *[LoCoMo | LongMemEval | Evo-Memory | MemoryBank | custom task suite | product telemetry | user study | other]*
Evaluation level: *[Component-level memory evaluation | system-level agent evaluation | model-level evaluation | product-level evaluation | other]*
Environment: *[Chat | tool use | browser/web | computer use | embodied environment | API workflow | offline dataset | synthetic benchmark | real-world deployment | other]*
Datasets or traces: *[Describe evaluation data, provenance, size, and whether it contains sensitive data.]*
Metrics: *[Accuracy | recall | precision | F1 | task success | user satisfaction | latency | cost | storage growth | hallucination rate | edit success | deletion success | other]*
Reproducibility: *[Code | configuration | seeds | datasets | prompts | model versions | hardware | logs | unavailable]*

Memory Quality

Capture quality: *[How accurately the system stores or internalizes salient information.]*
Update quality: *[How well the system handles new, conflicting, redundant, or stale information.]*
Use quality: *[How well the system surfaces or applies the right information at the right time.]*
Calibration: *[How well the system knows when it does not have relevant memory.]*
Grounding and provenance: *[How well outputs can be traced to stored memory, training data, source events, or model updates.]*

Efficiency and Scalability

Latency: *[Read, write, retrieval, update, fine-tuning, or inference latency.]*
Storage footprint: *[Memory size, index size, parameter delta size, cache size, growth behavior.]*
Compute cost: *[Embedding | retrieval | graph updates | model calls | training/fine-tuning | cache maintenance | other]*

Scale tested: *[Number of memories, users, sessions, documents, tokens, parameters, modalities, or tasks.]*

Degradation at scale: *[Known effects on quality, latency, cost, privacy, or maintainability as memory grows.]*

Utility Trade-offs

Performance trade-offs: *[Accuracy vs. latency | recall vs. context length | compression vs. fidelity | explicit memory vs. parametric generalization | update speed vs. stability | other]*

Operational trade-offs: *[Ease of deployment | maintainability | observability | interoperability | portability | vendor lock-in | other]*

Limitations and Risks

Technical Limitations

[Describe limitations in memory capture, storage, updating, retrieval, deletion, interpretability, scalability, modality support, or integration.]

Behavioral Risks

[Describe risks such as false memories, stale or conflicting memories, inappropriate personalization, overfitting to past behavior, undesired persistence, catastrophic forgetting, off-target parameter updates, or context pollution.]

Domain and User Limitations

[Describe domains, languages, modalities, user groups, or deployment settings where the system is not validated or may perform poorly.]

Recommendations

[Describe mitigations, monitoring practices, user education, review procedures, or additional evaluations recommended before deployment.]

Maintenance and Governance

Updates

Update cadence: `[More Information Needed]`

Backward compatibility: *[Describe compatibility across memory schemas, model versions, adapters, indexes, APIs, and stored states.]*

Migration support: *[Describe how existing memories are migrated, re-indexed, re-embedded, fine-tuned, or invalidated.]*

Monitoring

Runtime monitoring: *[Quality metrics | latency | cost | retrieval logs | write logs | deletion logs | privacy events | drift | other]*

Audit logs: *[Describe what is logged, who can access logs, and how long logs are retained.]*

User feedback channels: *[Corrections | deletion requests | reports | ratings | support tickets | other]*

Model and Dependency Information

Base models: *[Model names and versions, if any]*

Embedding models: *[Model names and versions, if any]*

Rerankers or classifiers: *[Model names and versions, if any]*

Databases and storage systems: *[Names and versions, if any]*

External services: *[APIs | cloud providers | analytics | monitoring | other]*

Privacy, Sovereignty, and Safety

This section appears after the core memory, performance, limitations, and governance sections, but it should still be filled carefully. For private agent memory systems, privacy is not only about whether data is secret. It also concerns whether information flows in ways that respect contextual norms, explicit consent, and user control over the data lifecycle.

Privacy Definition Used

Privacy framing: *[Describe the privacy theory or policy used by the system.]*

Suggested framing:

- **Contextual integrity:** Information should flow only in ways appropriate to the context, relationship, purpose, and user expectations.
- **Individual data sovereignty:** Users should have meaningful, verifiable visibility and control over data that concerns them, including the ability to inspect, correct, export, and delete it where technically possible.
- **Privacy tax:** The trade-off between memory-enabled performance gains and the privacy risks introduced by persistence, inference, retrieval, sharing, or internalization.

Data Sensitivity

Sensitive data categories: *[PII | health | finance | location | credentials | biometric data | minors' data | workplace data | private communications | inferred traits | other]*

Default handling of sensitive data: *[Stored | blocked | redacted | summarized | embedded | encrypted | ephemeral | user-confirmed | policy-dependent | unsupported]*

Inferred sensitive attributes: *[Describe whether the system can infer traits, preferences, health status, relationships, or other sensitive attributes from non-sensitive inputs.]*

Transparency

User inspection: *[Can users see what the system remembers or how memory influences outputs?]*

Completeness of inspection: *[All entries | selected entries | summaries only | metadata only | not inspectable | not applicable for parametric/latent memory]*

Provenance: *[Does the system track source events, documents, users, tools, timestamps, or transformations?]*

Inference tracing: *[Can derived memories or model behaviors be traced back to source memories, training examples, or update events?]*

Hidden state policy: *[Describe whether hidden memories, private profiles, latent states, or parameter updates can exist without user visibility.]*

User Control, Editability, and Erasability

Edit support: *[Direct edit | correction request | profile edit | entry overwrite | model update | unsupported | other]*

Deletion support: *[Entry deletion | session deletion | entity deletion | provenance deletion | account deletion | cache clearing | vector deletion | graph deletion | unlearning | unsupported]*

Deletion guarantees: *[Hard deletion | tombstone | soft deletion | removal from retrieval only | best-effort unlearning | retention in logs/backups | unsupported]*

Export or portability: *[Describe whether users can export memory content or metadata.]*

User authority: *[Describe whether authenticated user corrections override later agent, tool, or third-party inputs.]*

Contextual Integrity

Contextual integrity requires that information flow only in ways appropriate to the context in which it was shared, so that memories do not leak across personas, workspaces, origins, tenants, or other trust domains in ways that would violate contextual norms or user expectations.

Context boundaries: *[User | workspace | organization | tenant | persona | application | origin/domain | session | task | doc level | other]*

Boundary enforcement: *[Access control | namespace isolation | retrieval filters | policy engine | prompt constraints | encryption keys | model separation | unsupported | other]*

Cross-context sharing: *[Describe which information flows across contexts are appropriate, and under what conditions, consistent with contextual information norms and user expectations.]*

Low-trust content handling: *[Describe how the system prevents web pages, documents, tool outputs, or other low-trust sources from inducing information flows that violate contextual norms or extract out-of-scope memories.]*

Retention Policy Adherence

Retention policy support: *[User-defined | administrator-defined | environment-defined | fixed defaults | unsupported]*

Deny-list or do-not-remember rules: *[Describe whether prohibited information is blocked before storage or later removed.]*

Ephemeral contexts: *[Describe whether sensitive environments, private browsing modes, incognito sessions, or task-local contexts are excluded from durable memory.]*

Auto-expiration: *[Describe time-based deletion, decay, archival, or cache eviction policies.]*

Backup and log retention: *[Describe whether deleted or sensitive memory remains in logs, traces, analytics, backups, or model training data.]*

PII and Sensitive Information Handling

PII detection: *[Rules | classifiers | LLM judge | external DLP tool | user labels | unsupported]*

PII action: *[Block | redact | mask | hash | encrypt | summarize | ask user | store as-is | other]*

Structural memory after redaction: *[Describe whether the system retains non-sensitive structure after removing sensitive values.]*

Security and Adversarial Robustness

Threat model: *[Prompt injection | data poisoning | covert profiling | cross-origin extraction | malicious tools | insider access | account compromise | model inversion | training data extraction | other]*

Adversarial tests: *[Describe tests run against attempts to force retention, bypass deletion, extract context-restricted data, hide memories, or overwrite user edits.]*

Access controls: *[Authentication | authorization | tenant isolation | encryption | audit logging | rate limits | other]*

Incident response: *[Describe logging, detection, user notification, rollback, deletion, or remediation procedures.]*

Privacy Evaluation

Privacy benchmarks or audits: *[Transparency tests | mutation integrity tests | contextual integrity tests | retention adherence tests | adversarial tests | manual review | external audit | none]*

Known privacy risks: `[More Information Needed]`

Mitigations: `[More Information Needed]`

Residual risk: `[More Information Needed]`

Citation

If this memory system is described in a paper, report, or public artifact, cite it here.

BibTeX: `[More Information Needed]`

APA or plain-language citation: `[More Information Needed]`

Glossary

Memory system: A component or mechanism that preserves, transforms, retrieves, internalizes, or otherwise makes information from past interactions available to an agent or model over time.
Memory form: The representation that carries memory, such as raw text, structured records, vectors, graphs, latent states, caches, adapters, or model parameters.
Memory capability: The functional role memory plays, such as preserving user facts, maintaining task context, supporting personalization, recalling environment state, learning from past trajectories, or internalizing skills.
Memory lifecycle: The process by which information is captured, transformed, maintained, updated, forgotten, retrieved, or otherwise made available for use.
Formation: The process by which information from interactions, documents, traces, feedback, or training data becomes memory.
Evolution: The process by which memory changes over time through updating, consolidation, conflict resolution, pruning, forgetting, fine-tuning, or restructuring.
Retrieval or use: The process by which memory influences model or agent behavior, either through explicit retrieval, context injection, tool calls, latent reuse, or parameterized behavior.
Contextual integrity: A privacy framework in which information flows are appropriate when they align with contextual norms governing actors, attributes, transmission principles, and purposes.
Individual data sovereignty: The principle that users should have meaningful visibility and control over data that concerns them.
Privacy tax: The trade-off between the benefits produced by memory and the privacy risks introduced by storing, transforming, retrieving, sharing, or internalizing information.

More Information

[Add links to documentation, evaluations, issue trackers, examples, or contact channels.]

3 Structured Index Record

The Structured Index Record is the compact, machine-readable part of the AMS Card. It records identifying and capability attributes as typed fields and controlled values so that memory systems can be discovered, filtered, and compared without interpreting the narrative. Fields marked "one or more" are controlled multi-select fields rather than single-value classifications. The table below defines each field, its type or allowed values, and its purpose.

Field	Type or allowed values	Description
`memory_system_name`	Short text	The public name of the memory system, framework, service, adapter, database, or model variant.
`version`	Short text	The exact release, commit, model checkpoint, schema version, service version, or adapter version described by the card.
`developers`	List of names	The people or organizations responsible for developing the memory system.
`release_date`	Date (YYYY-MM-DD)	The date this version was released or made available.

continued on next page

Field	Type or allowed values	Description
`license`	License identifier or custom license	The software, model, data, or service license that governs use of the system. If licenses differ across components, explain that in the long version.

`homepage`
`repository`
`technical_reference`
`documentation`
`contact`
	URLs or contact points	Links or contact points for users who need implementation details, research context, support, or governance information.

`tags`	List of normalized labels, e.g. `memory-system`, `agent-memory`	Search and discovery labels. Use lowercase, hyphen-separated labels and normalize equivalent labels to one value rather than embedding descriptions in this field.
`memory_forms`	One or more of: raw-text | token-level | vector | graph | key-value | profile | latent | parametric | hybrid | other	The physical or computational form that carries memory.
`memory_functions`	One or more of: user facts | environment facts | preferences | experience traces | skills | strategies | working context | other	The main roles served by memory.
`modalities`	One or more of: text | image | audio | video | tool traces | browser state | structured data | other	The data modalities the system can capture, transform, store, internalize, retrieve, or otherwise use.
`supported_interfaces`	One or more of: API | SDK | MCP server | agent tool | model adapter | database backend | other	The integration surfaces through which agents, models, applications, or users interact with the memory system.
`evaluation_status`	Exactly one of: not evaluated | internally evaluated | externally evaluated | benchmarked	The level of evidence available for performance or utility claims.
`privacy_review_status`	Exactly one of: not reviewed | self-assessed | internally reviewed | independently reviewed	The level of review available for privacy, user control, contextual integrity, retention, and safety claims.